

Spring Boot Inventory Management System

Comprehensive System Design & Implementation Blueprint

Purpose: A professional planning document describing the requirements, architecture, database, APIs, user journeys, diagrams, and implementation roadmap for a Spring Boot application.

1. Project Overview

The proposed project is a simplified Inventory Management System for a small retail business. The application will allow authorized users to manage products, categories, suppliers, and stock levels through a RESTful Spring Boot backend. It will maintain inventory movement history and identify products that require restocking. The project is intentionally scoped to demonstrate professional backend concepts without introducing unnecessary complexity.

2. Objectives

- Centralize product and inventory management.
- Maintain accurate stock quantities and movement history.
- Provide secure REST APIs for a future web or mobile client.
- Apply maintainable Spring Boot and relational database practices.
- Provide a clear foundation for testing and future expansion.

3. Functional Requirements

ID	Requirement
FR-01	Users can register and log in securely.
FR-02	Authorized users can create, update, view, and delete products.
FR-03	Products can be grouped into categories.
FR-04	Supplier information can be maintained.
FR-05	Users can increase or decrease stock quantities.
FR-06	Every stock movement is recorded with quantity and timestamp.
FR-07	The system identifies products below their minimum stock threshold.
FR-08	The API returns clear validation and error responses.

4. Non-Functional Requirements

- Security: passwords are hashed and protected endpoints require authentication.
- Performance: normal CRUD operations should respond quickly for the expected workload.
- Maintainability: business, API, persistence, and security responsibilities remain separated.
- Reliability: important database operations use transactions.
- Scalability: modules can be extended without redesigning the whole system.

- Usability: APIs use predictable HTTP status codes and JSON response structures.

5. Proposed Technology Stack

Layer	Technology
Backend	Java + Spring Boot
API	Spring Web / REST
Security	Spring Security + JWT
Persistence	Spring Data JPA / Hibernate
Database	PostgreSQL
Validation	Jakarta Bean Validation
Testing	JUnit + Mockito + Spring Boot Test
Documentation	OpenAPI / Swagger
Build	Maven
Version Control	Git + GitHub

6. System Architecture

A layered monolithic architecture is recommended for the first version. The client communicates with REST controllers. Controllers validate requests and delegate business logic to services. Services enforce business rules and coordinate repositories. Repositories communicate with PostgreSQL through JPA/Hibernate. Spring Security filters validate JWT tokens before protected requests reach application logic.

```
Client (Web / Mobile)
|
v
REST Controllers
|
v
Service Layer
|
v
Repository Layer
|
v
PostgreSQL
```

JWT Authentication -> Spring Security Filter -> Protected Controllers

7. Suggested Project Structure

```
src/main/java/com/example/inventory/
■■■ controller/
■■■ service/
■■■ repository/
■■■ entity/
■■■ dto/
■■■ security/
■■■ exception/
■■■ InventoryApplication.java
```

8. Database Design

Table	Important Fields	Purpose
-------	------------------	---------

users	id, name, email, password, role	Authenticated users
categories	id, name, description	Product grouping
suppliers	id, name, email, phone	Supplier details
products	id, name, sku, price, quantity, min_stock, category_id, supplier_id	Inventory items
stock_movements	id, product_id, type, quantity, created_at, user_id	Stock change history

Relationships: one category can contain many products; one supplier can supply many products; one product can have many stock movements; one user can create many stock movement records.

9. Data Flow Diagram

LOGIN:

User -> Auth Controller -> Security Service -> User Database -> JWT Token

PRODUCT:

User -> JWT Filter -> Product Controller -> Product Service -> Repository -> PostgreSQL -> JSON Response

STOCK:

User -> Stock Controller -> Stock Service -> Product Quantity Update
-> Stock Movement Record -> PostgreSQL

10. API Endpoint Summary

Method	Endpoint	Purpose
POST	/api/auth/register	Register a user
POST	/api/auth/login	Authenticate and receive JWT
GET	/api/products	List products
GET	/api/products/{id}	Get product details
POST	/api/products	Create product
PUT	/api/products/{id}	Update product
DELETE	/api/products/{id}	Delete product
GET	/api/categories	List categories
POST	/api/categories	Create category
GET	/api/suppliers	List suppliers
POST	/api/suppliers	Create supplier
POST	/api/products/{id}/stock	Increase/decrease stock
GET	/api/products/low-stock	Find low-stock products
GET	/api/products/{id}/movements	View stock history

11. User Journeys

Login: User enters credentials → API authenticates → JWT is returned → protected features become available.

Add Product: User submits product details → validation runs → service saves the product → created product is returned.

Update Stock: User selects a product → submits movement type and quantity → service validates the operation → quantity and movement history are updated.

Check Low Stock: User requests low-stock products → service compares current quantity with minimum threshold → matching products are returned.

12. Error Handling

The API should use consistent HTTP responses: 200 OK for successful retrieval/update, 201 Created for successful creation, 400 Bad Request for validation errors, 401 Unauthorized for invalid authentication, 403 Forbidden for insufficient permissions, 404 Not Found for missing resources, and 500 Internal Server Error for unexpected failures. A global exception handler should return a consistent JSON error format.

13. Security Design

- Use BCrypt or an equivalent password hashing mechanism.
- Use JWT access tokens for stateless authentication.
- Protect inventory endpoints with Spring Security.
- Validate request bodies and path parameters.
- Never expose passwords in API responses.

- Keep database credentials and JWT secrets outside source control.

14. Implementation Roadmap

Phase	Work	Deliverable
1	Requirements + setup	Project skeleton, Git repository, requirements
2	Database + entities	PostgreSQL schema and JPA entities
3	Authentication	Register/login, password hashing, JWT
4	Core modules	Product, category, supplier CRUD APIs
5	Inventory operations	Stock update and movement history
6	Documentation	Global exceptions and Swagger/OpenAPI
7	Testing	Unit and integration tests
8	Final review	Bug fixes, API verification, final documentation

15. Testing Strategy

Testing will be performed at multiple levels. Unit tests will verify service and business logic in isolation. Controller tests will verify request validation, HTTP status codes, and response structures. Integration tests will verify interaction between Spring Boot, repositories, security, and the database. Swagger or Postman can be used for manual API verification. Important test cases include invalid login, duplicate SKU, missing product, invalid stock quantity, unauthorized access, and successful CRUD operations.

16. Risks and Mitigation

Risk	Mitigation
Incorrect stock calculations	Use transactional service methods and validate movement quantities.
Unauthorized access	Apply Spring Security and role-based endpoint protection.
Database inconsistency	Use foreign keys, constraints, and transactions.
Scope becoming too large	Keep the first release modular but monolithic and avoid unnecessary features.
Insufficient testing	Test each module as it is implemented.

17. Minimum 200-Word Project Report

The Spring Boot Inventory Management System is proposed as a backend application for a small retail business that needs a simple and reliable way to manage its inventory. The primary purpose of the project is to replace basic manual inventory tracking with a centralized REST-based system. The application will allow authorized users to authenticate, manage products, organize products into categories, maintain supplier information, and update stock quantities. Every stock adjustment will be recorded as an inventory movement so users can understand how product quantities changed over time. The system will also provide a low-stock operation that identifies products whose current quantity has reached or fallen below the configured minimum level.

The application will use Java and Spring Boot, with Spring Web for REST APIs, Spring Security and JWT for authentication, Spring Data JPA and Hibernate for persistence, and PostgreSQL as the relational database. A layered monolithic architecture will be used because it keeps the first version simple while separating controllers, services, repositories, entities, DTOs, security, and exception handling. This structure also provides a useful foundation for future expansion. Non-functional requirements include security, maintainability, reliability, validation, predictable API responses, and reasonable performance for a

small-business workload.

Development will be divided into phases covering requirements, project setup, database design, authentication, CRUD modules, inventory operations, documentation, testing, and final review. Testing will include unit tests, controller tests, integration tests, and manual API verification. Overall, this project demonstrates the complete software planning process from requirements analysis and system design to implementation scheduling. The document provides a clear blueprint that can guide coding and debugging while reducing unnecessary architectural changes during development.

18. Conclusion

This design establishes a practical blueprint for developing the Inventory Management System. The proposed architecture separates API handling, business logic, persistence, and security concerns. The database model supports the inventory workflow while preserving movement history, and the API plan provides a clear contract for future clients. The phased roadmap makes implementation manageable and gives each development stage a measurable deliverable.